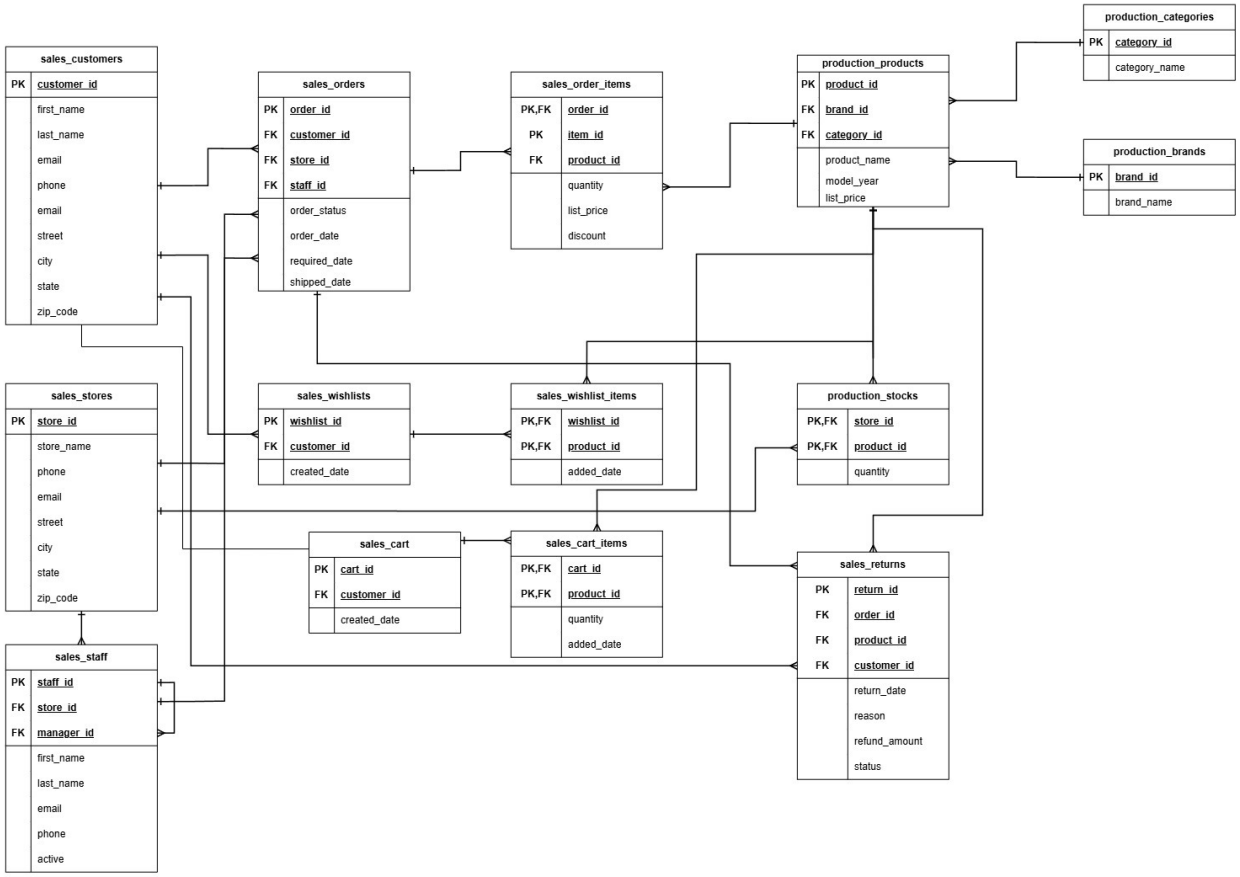
Group Project – Group 4

- Alexander Bouwhuis
- Yuri Koller
- Yuhong Lin
- Roney Thomas Mannoocheril



(*Diagram is bit hard to read here, included as separate image file in submission zip file.*)

ER Diagram Description

Overall System Description

This database represents a bike store sales management system designed to manage customers, orders, products, inventory, and extended business features such as wishlists, shopping carts, and returns.

The system integrates both operational data (customers, orders, staff) and product management data (brands, categories, inventory), providing a comprehensive structure for retail operations.

Core Entities

Customers and Orders

The sales_customers entity stores customer information such as name, contact details, and address. The sales_orders entity represents purchase transactions and links customers, stores, and staff.

- One customer can place many orders (1:M)
- Each order belongs to one customer

Order Details

The sales_order_items table is an associative entity that connects orders and products.

- One order can contain multiple products
- One product can appear in multiple orders

This resolves a many-to-many (M:N) relationship between orders and products.

Products and Classification

The production_products entity stores product information and is linked to both brands and categories.

- One brand has many products (1:M)
- One category has many products (1:M)

Inventory Management

The production_stocks table represents inventory levels of products in stores.

- One store can stock many products
- One product can be available in many stores

Stores and Staff

The sales_stores entity stores store information. The sales_staff entity stores employee information.

- One store employs many staff members (1:M)

Recursive Relationship (Staff Hierarchy)

The sales_staff entity includes a foreign key manager_id that references staff_id in the same table.

- One manager can supervise multiple staff members (1:M)
- Each staff member reports to one manager

Extended Features

Wishlist System

The sales_wishlists entity stores customer wishlists. The sales_wishlist_items table connects wishlists and products.

- One customer can have multiple wishlists (1:M)
- One wishlist can contain multiple products

Shopping Cart System

The shopping cart feature allows customers to temporarily store products before completing a purchase.

Entities:

- sales_cart
- sales_cart_items

Relationships:

- One customer can have one active cart (1:1)
- One cart can contain multiple items (1:M)
- Each cart item references one product (M:1)

Implementation Details: The sales_cart table stores cart-level information, including customer_id and created_date. The sales_cart_items table uses a composite primary key (cart_id, product_id) to ensure each product appears only once per cart. Quantity is stored at the cart item level to support multiple units of the same product.

Returns and Refunds

The sales_returns entity manages returned items and refund processing.

- Each return is linked to one order
- Each return is linked to one product
- Each return is linked to one customer

Key Design Concepts

Foreign keys enforce relationships and maintain referential integrity. Many-to-many relationships are resolved using associative tables. Primary keys ensure uniqueness. The design supports scalability and future extensions.

Conclusion

This ER diagram models a real-world retail system and ensures data integrity, scalability, and clear relationships.

10 Business Requirement Queries Implemented

1. Search products by category
2. Manage shopping cart
3. Register/unregister user
4. Track shipping status
5. Manage vendors
6. Monthly sales report
7. Most popular products
8. Customer tiers by purchase history
9. Wish list items
10. Returns and refunds

1) Search products by category, Returns all products in a given category with brand info. Change the category_name value to search a different category.

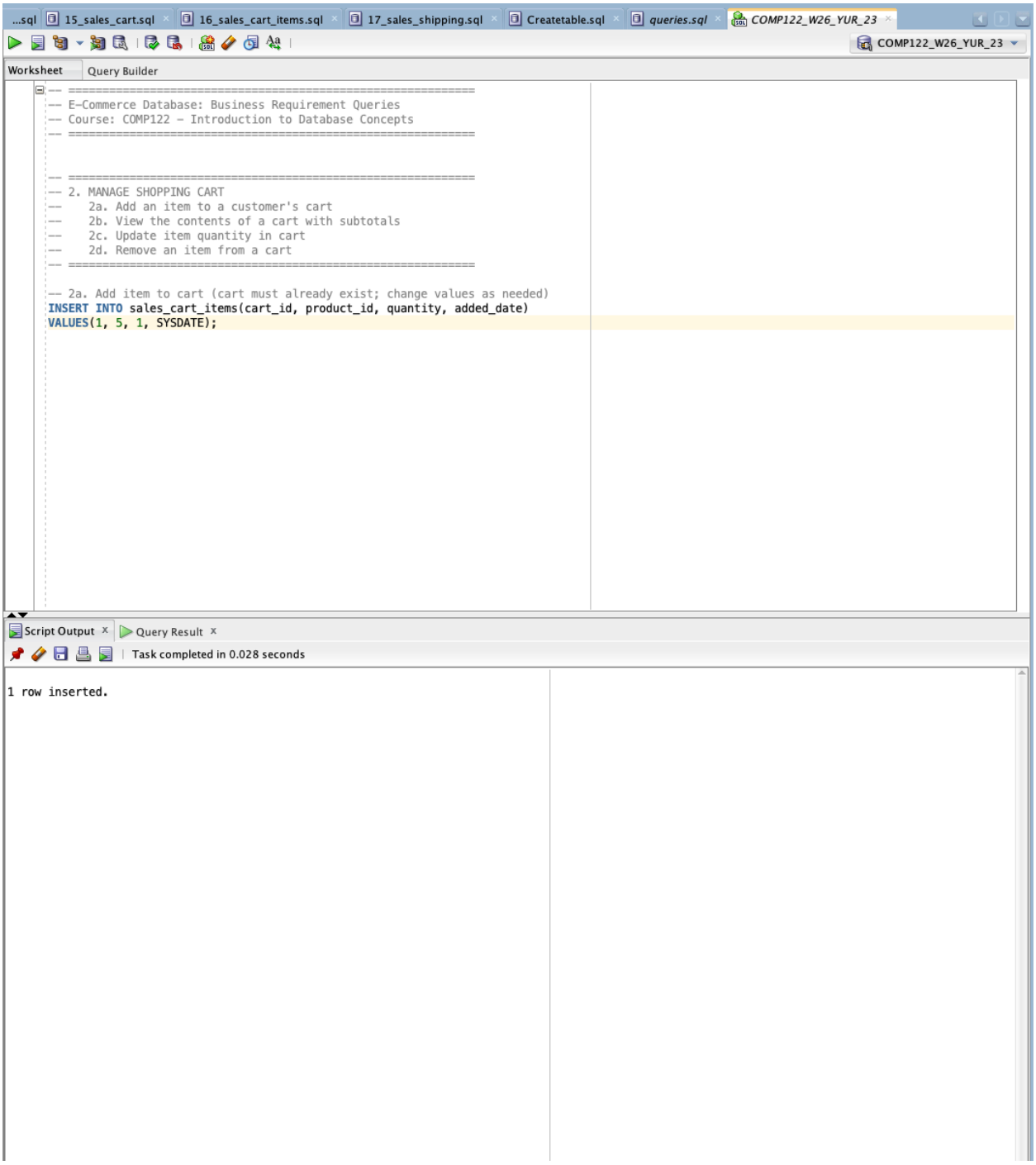
```sql
-- ============================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ============================================================


-- ============================================================
-- 1. SEARCH PRODUCTS BASED ON CATEGORY
--    Returns all products in a given category with brand info.
--    Change the category_name value to search a different category.
-- ============================================================
SELECT
    p.product_id,
    p.product_name,
    c.category_name,
    b.brand_name,
    p.model_year,
    p.list_price
FROM production_products p
JOIN production_categories c ON p.category_id = c.category_id
JOIN production_brands     b ON p.brand_id     = b.brand_id
WHERE c.category_name = 'Mountain Bikes'
ORDER BY p.list_price;
```

All Rows Fetched: 8 in 0.04 seconds

	PRODUCT_ID	PRODUCT_NAME	CATEGORY_NAME	BRAND_NAME	MODEL_YEAR	LIST_PRICE
1	9	Scott Aspect 950	Mountain Bikes	Scott	2023	529.99
2	5	Giant Talon 3	Mountain Bikes	Giant	2023	599.99
3	7	Cannondale Trail 8	Mountain Bikes	Cannondale	2024	749.99
4	1	Trek Marlin 7	Mountain Bikes	Trek	2023	849.99
5	3	Specialized Rockhopper	Mountain Bikes	Specialized	2024	1199.99
6	15	Cube Stereo 120	Mountain Bikes	Cube	2024	1349.99
7	20	Cube Reaction Race	Mountain Bikes	Cube	2024	1699.99
8	12	Santa Cruz Highball	Mountain Bikes	Santa Cruz	2024	2299.99

2) Manage shopping cart

a. Add an item to a customer's cart



b. View the contents of a cart with subtotals

```sql
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 2. MANAGE SHOPPING CART
--    2a. Add an item to a customer's cart
--    2b. View the contents of a cart with subtotals
--    2c. Update item quantity in cart
--    2d. Remove an item from a cart
-- ==========================================================

-- 2b. View cart for customer_id = 2
SELECT
    ci.cart_id,
    c.first_name || ' ' || c.last_name      AS customer_name,
    p.product_id,
    p.product_name,
    ci.quantity,
    p.list_price,
    ci.quantity * p.list_price               AS subtotal
FROM sales_cart        sc
JOIN sales_customers    c  ON sc.customer_id = c.customer_id
JOIN sales_cart_items   ci ON sc.cart_id     = ci.cart_id
JOIN production_products p  ON ci.product_id = p.product_id
WHERE sc.customer_id = 2
ORDER BY ci.added_date;
```

All Rows Fetched: 3 in 0.029 seconds

	CART_ID	CUSTOMER_NAME	PRODUCT_ID	PRODUCT_NAME	QUANTITY	LIST_PRICE	SUBTOTAL
1	1	Kasha Todd	3	Specialized Rockhopper	1	1199.99	1199.99
2	1	Kasha Todd	16	Trek Powerfly 4	1	2999.99	2999.99
3	1	Kasha Todd	5	Giant Talon 3	1	599.99	599.99

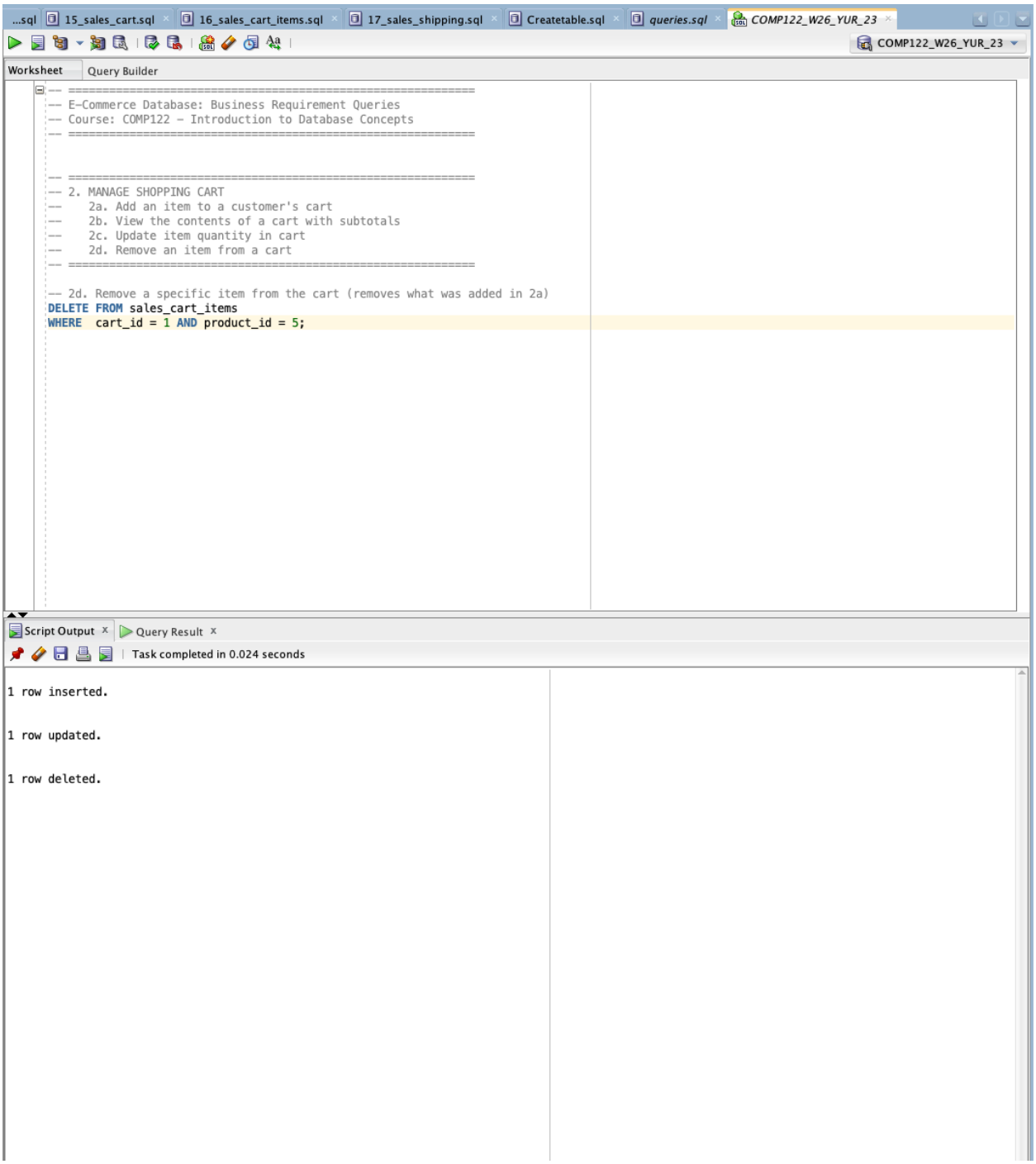
c. Update item quantity in cart

```
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 2. MANAGE SHOPPING CART
--    2a. Add an item to a customer's cart
--    2b. View the contents of a cart with subtotals
--    2c. Update item quantity in cart
--    2d. Remove an item from a cart
-- ==========================================================

-- 2c. Update quantity of a product already in the cart
UPDATE sales_cart_items
SET    quantity = 2
WHERE  cart_id = 1 AND product_id = 3;
```

Task completed in 0.035 seconds

```
1 row inserted.

1 row updated.
```

d. Remove an item from a cart



3) Register/Unregister User

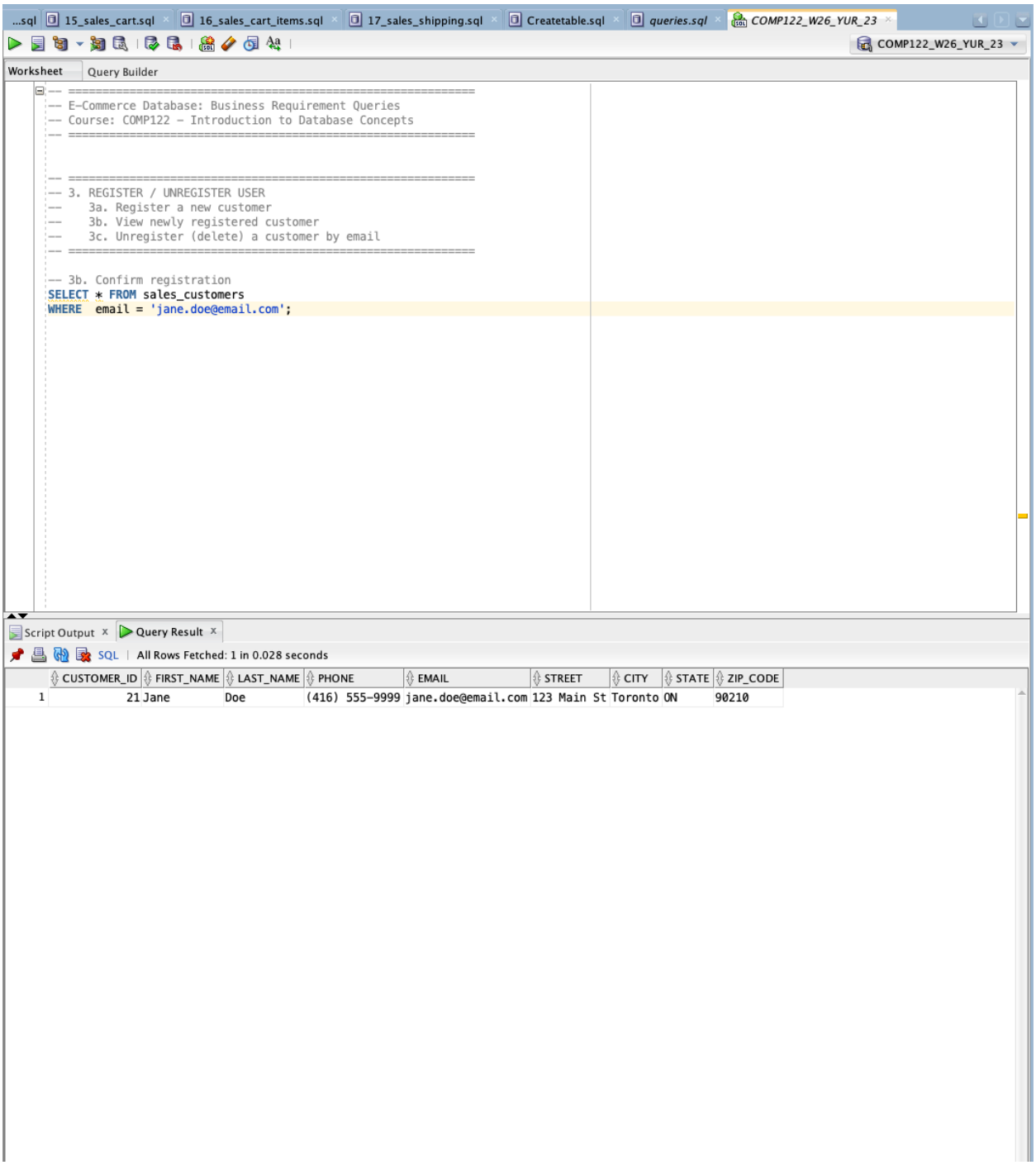
a. Register a new customer

```sql
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 3. REGISTER / UNREGISTER USER
--    3a. Register a new customer
--    3b. View newly registered customer
--    3c. Unregister (delete) a customer by email
-- ==========================================================

-- 3a. Register new customer
INSERT INTO sales_customers(first_name, last_name, phone, email, street, city, state, zip_code)
VALUES('Jane', 'Doe', '(416) 555-9999', 'jane.doe@email.com', '123 Main St', 'Toronto', 'ON', '90210');
```

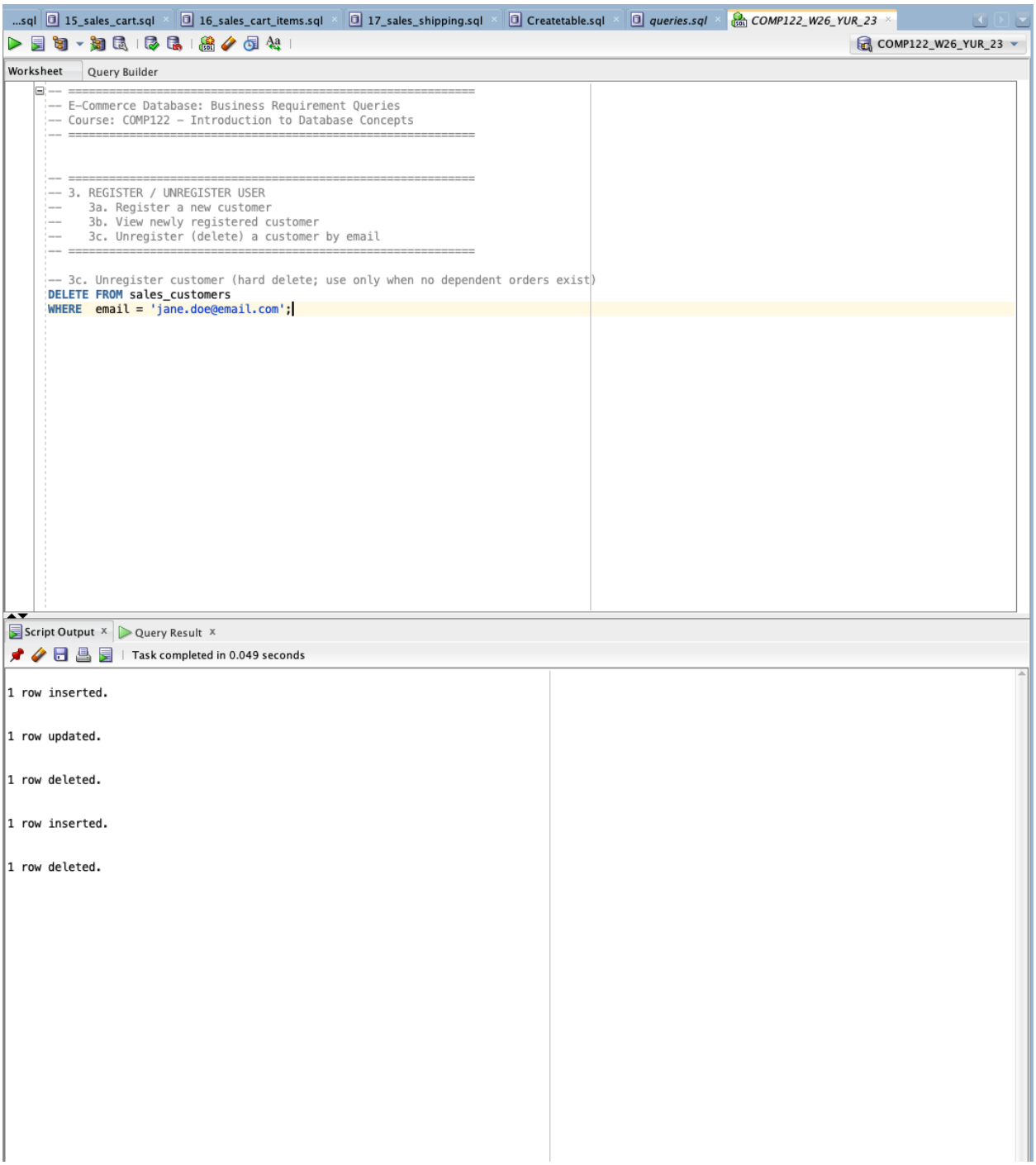
Task completed in 0.035 seconds

```
1 row inserted.

1 row updated.

1 row deleted.

1 row inserted.
```

b. View newly registered customer



c. Unregister (delete) a customer by email



4) Track shipping status. Shows the shipping/tracking details for every order belonging to a specific customer.

```sql
-- ============================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ============================================================


-- ============================================================
-- 4. TRACK SHIPPING STATUS
--    Shows the shipping/tracking details for every order
--    belonging to a specific customer.
-- ============================================================
SELECT
    o.order_id,
    c.first_name || ' ' || c.last_name   AS customer_name,
    o.order_date,
    o.required_date,
    s.carrier,
    s.tracking_number,
    s.shipping_status,
    s.estimated_delivery,
    s.actual_delivery
FROM sales_orders    o
JOIN sales_customers  c ON o.customer_id  = c.customer_id
JOIN sales_shipping   s ON o.order_id     = s.order_id
WHERE o.customer_id = 1
ORDER BY o.order_date DESC;
```

All Rows Fetched: 2 in 0.026 seconds

	ORDER_ID	CUSTOMER_NAME	ORDER_DATE	REQUIRED_DATE	CARRIER	TRACKING_NUMBER	SHIPPING_STATUS	ESTIMATED_DELIVERY	ACTUAL_DELIVERY
1	21	Debra Burks	20231101	20231108	UPS	UPS1Z999AA10789012340	Delivered	20231108	20231104
2	1	Debra Burks	20230105	20230110	UPS	UPS1Z999AA10123456784	Delivered	20230110	20230108

5) Manage Vendors

a. Add a new vendor

```sql
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 5. MANAGE VENDORS
--    5a. Add a new vendor
--    5b. List all active vendors with their supplied brands
--    5c. Deactivate a vendor (soft delete)
-- ==========================================================

-- 5a. Add vendor
INSERT INTO production_vendors(vendor_id, vendor_name, contact_name, phone, email, city, state, active)
VALUES(11, 'Pacific Cycle Imports', 'Amy Tran', '(604) 555-2001', 'amy@pacificcycle.com', 'Vancouver', 'BC', 1);
```

```
Task completed in 0.036 seconds

1 row inserted.

1 row updated.

1 row deleted.

1 row inserted.

1 row deleted.

1 row inserted.
```

b. List all active vendors with their supplied brands

```sql
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 5. MANAGE VENDORS
--    5a. Add a new vendor
--    5b. List all active vendors with their supplied brands
--    5c. Deactivate a vendor (soft delete)
-- ==========================================================

-- 5b. List active vendors with the brands they supply
SELECT
    v.vendor_id,
    v.vendor_name,
    v.contact_name,
    v.phone,
    v.email,
    v.city,
    v.state,
    b.brand_name
FROM production_vendors       v
JOIN production_vendor_brands vb ON v.vendor_id = vb.vendor_id
JOIN production_brands        b  ON vb.brand_id  = b.brand_id
WHERE v.active = 1
ORDER BY v.vendor_name;
```

All Rows Fetched: 14 in 0.032 seconds

	VENDOR_ID	VENDOR_NAME	CONTACT_NAME	PHONE	EMAIL	CITY	STATE	BRAND_NAME
1	1	BikeSupply Co	James Carter	212-555-0101	james.carter@bikesupply.com	New York	NY	Specialized
2	1	BikeSupply Co	James Carter	212-555-0101	james.carter@bikesupply.com	New York	NY	Trek
3	2	CycleParts Inc	Laura Chen	310-555-0202	laura.chen@cycleparts.com	Los Angeles	CA	Specialized
4	2	CycleParts Inc	Laura Chen	310-555-0202	laura.chen@cycleparts.com	Los Angeles	CA	Giant
5	4	GearUp Distributors	Sara Kim	713-555-0404	sara.kim@gearup.com	Houston	TX	Bianchi
6	4	GearUp Distributors	Sara Kim	713-555-0404	sara.kim@gearup.com	Houston	TX	Santa Cruz
7	7	PedalPro Imports	Tom Brooks	210-555-0707	tom.brooks@pedalpro.com	San Antonio	TX	Giant
8	7	PedalPro Imports	Tom Brooks	210-555-0707	tom.brooks@pedalpro.com	San Antonio	TX	Trek
9	5	SpeedLine Wholesale	Paul Jordan	602-555-0505	paul.jordan@speedline.com	Phoenix	AZ	Cervelo
10	5	SpeedLine Wholesale	Paul Jordan	602-555-0505	paul.jordan@speedline.com	Phoenix	AZ	Orbea
11	8	TrailBlaze Suppliers	Amy Walsh	619-555-0808	amy.walsh@trailblaze.com	San Diego	CA	Scott
12	8	TrailBlaze Suppliers	Amy Walsh	619-555-0808	amy.walsh@trailblaze.com	San Diego	CA	Orbea
13	3	WheelWorld Ltd	Mark Evans	312-555-0303	mark.evans@wheelworld.com	Chicago	IL	Scott
14	3	WheelWorld Ltd	Mark Evans	312-555-0303	mark.evans@wheelworld.com	Chicago	IL	Cannondale

c. Deactivate a vendor (soft delete)

```
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 5. MANAGE VENDORS
--    5a. Add a new vendor
--    5b. List all active vendors with their supplied brands
--    5c. Deactivate a vendor (soft delete)
-- ==========================================================

-- 5c. Deactivate vendor (does not delete historical data)
UPDATE production_vendors
SET    active = 0
WHERE  vendor_id = 10;
```

Task completed in 0.04 seconds

```
1 row inserted.

1 row updated.

1 row deleted.

1 row inserted.

1 row deleted.

1 row inserted.

1 row updated.
```

6) Generate monthly sales report. Aggregates completed orders by month.

```sql
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 6. GENERATE MONTHLY SALES REPORT
--    Aggregates completed orders (status = 4) by month,
--    showing order count and total revenue.
-- ==========================================================
SELECT
    TO_CHAR(o.order_date, 'YYYY-MM')                          AS sales_month,
    COUNT(DISTINCT o.order_id)                                AS total_orders,
    SUM(oi.quantity)                                          AS units_sold,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)), 2) AS total_revenue
FROM sales_orders       o
JOIN sales_order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = 4
GROUP BY TO_CHAR(o.order_date, 'YYYY-MM')
ORDER BY sales_month;
```

All Rows Fetched: 15 in 0.028 seconds

	SALES_MONTH	TOTAL_ORDERS	UNITS_SOLD	TOTAL_REVENUE
1	2023-01	2	3	2802.47
2	2023-02	1	2	1489.98
3	2023-03	2	3	4153.47
4	2023-04	1	2	1179.98
5	2023-05	1	1	712.49
6	2023-06	2	3	6354.97
7	2023-07	1	1	2603.99
8	2023-08	2	3	3222.47
9	2023-09	1	2	1139.98
10	2023-10	1	2	1099.98
11	2023-11	2	4	5457.96
12	2023-12	2	3	4930.97
13	2024-01	2	3	5951.97
14	2024-02	1	2	2014.98
15	2024-03	1	2	1369.98

7) Find the most popular products

```sql
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 7. IDENTIFY THE MOST POPULAR PRODUCTS
--    Ranks products by total units sold across all orders.
-- ==========================================================
SELECT
    p.product_id,
    p.product_name,
    c.category_name,
    b.brand_name,
    SUM(oi.quantity)                                        AS total_units_sold,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)), 2) AS total_revenue
FROM sales_order_items   oi
JOIN production_products  p  ON oi.product_id  = p.product_id
JOIN production_categories c ON p.category_id  = c.category_id
JOIN production_brands     b ON p.brand_id     = b.brand_id
GROUP BY p.product_id, p.product_name, c.category_name, b.brand_name
ORDER BY total_units_sold DESC
FETCH FIRST 10 ROWS ONLY;
```

All Rows Fetched: 10 in 0.037 seconds

	PRODUCT_ID	PRODUCT_NAME	CATEGORY_NAME	BRAND_NAME	TOTAL_UNITS_SOLD	TOTAL_REVENUE
1	5	Giant Talon 3	Mountain Bikes	Giant	4	2309.96
2	19	Specialized Grom 20	Kids Bikes	Specialized	3	839.97
3	2	Trek Domane AL 2	Road Bikes	Trek	3	2399.97
4	6	Giant Escape 3	Hybrid Bikes	Giant	3	1399.97
5	4	Specialized Sirrus 3.0	Hybrid Bikes	Specialized	2	1799.98
6	7	Cannondale Trail 8	Mountain Bikes	Cannondale	2	1462.48
7	11	Bianchi Intenso	Road Bikes	Bianchi	2	3457.98
8	1	Trek Marlin 7	Mountain Bikes	Trek	2	1614.98
9	12	Santa Cruz Highball	Mountain Bikes	Santa Cruz	2	4484.98
10	10	Scott Speedster 40	Road Bikes	Scott	2	1329.98

8) Categorize customers based on purchase history

```sql
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 8. CATEGORIZE CUSTOMERS BASED ON PURCHASE HISTORY
--    Segments customers into tiers based on lifetime spend:
--      Platinum: >= $5,000  |  Gold: >= $2,000
--      Silver:   >= $500    |  Bronze: < $500
-- ==========================================================
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name                          AS customer_name,
    c.email,
    COUNT(DISTINCT o.order_id)                                  AS total_orders,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)), 2) AS lifetime_spend,
    CASE
        WHEN SUM(oi.quantity * oi.list_price * (1 - oi.discount)) >= 5000 THEN 'Platinum'
        WHEN SUM(oi.quantity * oi.list_price * (1 - oi.discount)) >= 2000 THEN 'Gold'
        WHEN SUM(oi.quantity * oi.list_price * (1 - oi.discount)) >= 500  THEN 'Silver'
        ELSE 'Bronze'
    END                                                         AS customer_tier
FROM sales_customers    c
JOIN sales_orders       o  ON c.customer_id = o.customer_id
JOIN sales_order_items  oi ON o.order_id    = oi.order_id
WHERE o.order_status = 4
GROUP BY c.customer_id, c.first_name, c.last_name, c.email
ORDER BY lifetime_spend DESC;
```

All Rows Fetched: 17 in 0.04 seconds

	CUSTOMER_ID	CUSTOMER_NAME	EMAIL	TOTAL_ORDERS	LIFETIME_SPEND	CUSTOMER_TIER
1	11	Ethan Brooks	ethan.brooks@yahoo.com	2	5879.98	Platinum
2	3	Tameka Fisher	tameka.fisher@hotmail.com	2	5199.95	Platinum
3	13	Noah Patel	noah.patel@hotmail.com	1	3291.98	Gold
4	1	Debra Burks	debra.burks@gmail.com	2	3155.47	Gold
5	12	Olivia Kim	olivia.kim@gmail.com	1	3134.98	Gold
6	7	Mireille Hahn	mireille.hahn@yahoo.com	1	2849.99	Gold
7	5	Charolette Rice	charolette.rice@gmail.com	1	2849.99	Gold
8	9	Carlos Mendez	carlos.mendez@outlook.com	2	2793.47	Gold
9	14	Ava Johnson	ava.johnson@gmail.com	1	2603.99	Gold
10	19	James Young	james.young@gmail.com	2	2469.96	Gold
11	15	Liam Turner	liam.turner@gmail.com	1	2414.98	Gold
12	17	Mason Walker	mason.walker@gmail.com	1	2014.98	Gold
13	2	Kasha Todd	kasha.todd@yahoo.com	1	1394.99	Silver
14	6	Shelly Wade	shelly.wade@gmail.com	1	1303.48	Silver
15	8	Layla Hassan	layla.hassan@gmail.com	1	1179.98	Silver
16	18	Isabella Hall	isabella.hall@outlook.com	1	1139.98	Silver
17	16	Sophia Evans	sophia.evans@yahoo.com	1	807.49	Silver

9) List all items on a user's wish list

```sql
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 9. LIST ALL ITEMS ON A USER'S WISH LIST
--    Returns all wish-listed products for a given customer.
--    Change the WHERE clause customer_id to target another user.
-- ==========================================================
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name  AS customer_name,
    p.product_name,
    cat.category_name,
    b.brand_name,
    p.list_price,
    wi.added_date
FROM sales_wishlists      wl
JOIN sales_wishlist_items wi  ON wl.wishlist_id = wi.wishlist_id
JOIN production_products  p   ON wi.product_id  = p.product_id
JOIN production_categories cat ON p.category_id = cat.category_id
JOIN production_brands     b   ON p.brand_id    = b.brand_id
JOIN sales_customers       c   ON wl.customer_id = c.customer_id
WHERE wl.customer_id = 1
ORDER BY wi.added_date DESC;
```

All Rows Fetched: 2 in 0.037 seconds

	CUSTOMER_ID	CUSTOMER_NAME	PRODUCT_NAME	CATEGORY_NAME	BRAND_NAME	LIST_PRICE	ADDED_DATE
1	1	Debra Burks	Santa Cruz Highball	Mountain Bikes	Santa Cruz	2299.99	20230212
2	1	Debra Burks	Specialized Rockhopper	Mountain Bikes	Specialized	1199.99	20230211

10) Handle returns and refunds

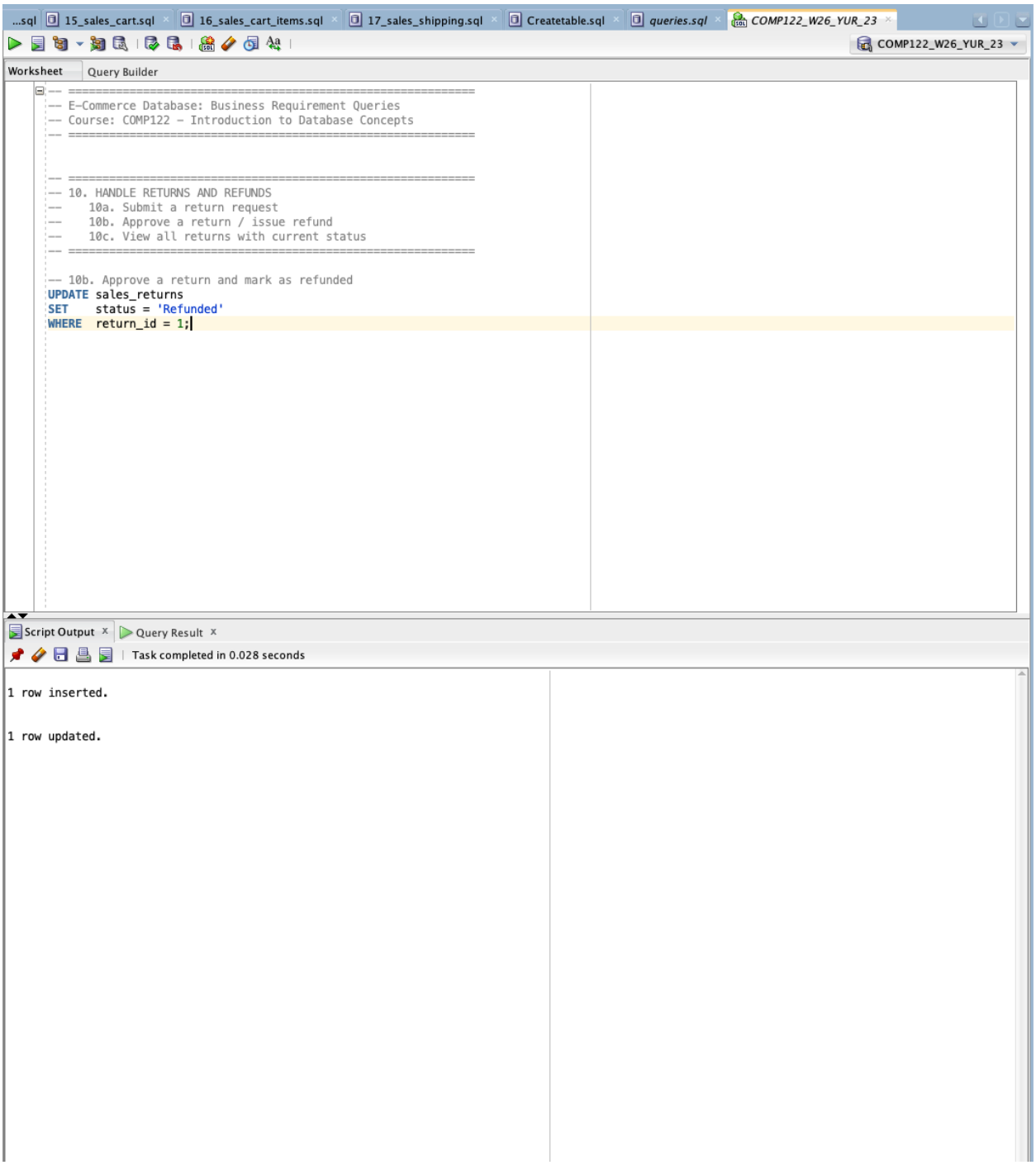
a. Submit a return request

```
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 10. HANDLE RETURNS AND REFUNDS
--    10a. Submit a return request
--    10b. Approve a return / issue refund
--    10c. View all returns with current status
-- ==========================================================

-- 10a. Submit a return request
INSERT INTO sales_returns(return_id, order_id, product_id, customer_id, return_date, reason, refund_amount, status)
VALUES(11, 22, 19, 3, SYSDATE, 'Defective product', 279.99, 'Pending');
```

Task completed in 0.034 seconds

```
1 row inserted.
```

b. Approve a return / issue refund



c. View all returns with current status

```sql
-- ==========================================================
-- E-Commerce Database: Business Requirement Queries
-- Course: COMP122 - Introduction to Database Concepts
-- ==========================================================


-- ==========================================================
-- 10. HANDLE RETURNS AND REFUNDS
--     10a. Submit a return request
--     10b. Approve a return / issue refund
--     10c. View all returns with current status
-- ==========================================================

-- 10c. View all returns with customer and product details
SELECT
    r.return_id,
    c.first_name || ' ' || c.last_name  AS customer_name,
    p.product_name,
    r.return_date,
    r.reason,
    r.refund_amount,
    r.status
FROM sales_returns       r
JOIN sales_customers     c ON r.customer_id = c.customer_id
JOIN production_products p ON r.product_id  = p.product_id
ORDER BY r.return_date DESC;
```

All Rows Fetched: 11 in 0.032 seconds

	RETURN_ID	CUSTOMER_NAME	PRODUCT_NAME	RETURN_DATE	REASON	REFUND_AMOUNT	STATUS
1	11	Tameka Fisher	Specialized Grom 20	20260409	Defective product	279.99	Pending
2	10	Carlos Mendez	Cube Reaction Race	20231230	Not as described	1699.99	Rejected
3	9	Debra Burks	Bianchi Intenso	20231118	Changed mind	1899.99	Approved
4	8	Sophia Evans	Trek Marlin 7	20230906	Wrong size	849.99	Pending
5	7	Ava Johnson	Orbea Orca M30	20230805	Defective product	2799.99	Approved
6	6	Ethan Brooks	Cervelo R3	20230625	Damaged in shipping	3499.99	Approved
7	5	Layla Hassan	Specialized Grom 20	20230501	Wrong color	279.99	Rejected
8	4	Shelly Wade	Scott Aspect 950	20230325	Not as described	529.99	Approved
9	3	Tameka Fisher	Specialized Rockhopper	20230218	Changed mind	1199.99	Pending
10	2	Kasha Todd	Cannondale CAAD13	20230128	Defective product	1499.99	Approved
11	1	Debra Burks	Trek Marlin 7	20230120	Wrong size	849.99	Refunded